

DECISIONES TÉCNICAS

1. Patrón de arquitectura: MVVM

Se adoptó el patrón Model-View-ViewModel (MVVM) como arquitectura base de la aplicación móvil. El Model corresponde a las interfaces TypeScript definidas en `app/models/` y los servicios en `app/services/`. La View son las plantillas XML de cada pantalla. El ViewModel son las clases Observable en cada archivo `view-model.ts` que exponen el estado y manejan la lógica de la interfaz.

Esta decisión se tomó porque NativeScript soporta data binding bidireccional nativo entre XML y clases Observable, lo que elimina la necesidad de actualizar manualmente la interfaz de usuario. Cada pantalla tiene su propio ViewModel, manteniendo la separación de responsabilidades.

2. Lenguaje: TypeScript sobre JavaScript

Se utilizó TypeScript 5.4 en lugar de JavaScript plano por las siguientes razones: detección de errores en tiempo de desarrollo y no en runtime, autocompletado de APIs nativas de Android gracias al sistema de metadatos de NativeScript, interfaces explícitas para modelos (Delivery, Client, Driver, CreateDeliveryRequest) que documentan la estructura esperada de la API, y mayor mantenibilidad al trabajar en equipo.

3. Organización modular por pantalla

Cada funcionalidad se encapsula en su propia carpeta: login para autenticación, home para el listado de entregas, delivery-create para creación de entregas, edit-delivery para edición, clients para registro de clientes y drivers para registro de repartidores. Cada módulo contiene únicamente los tres archivos que le corresponden: el archivo de página, el XML de vista y el view-model con la lógica. Esto facilita la navegación del código y el trabajo paralelo entre integrantes del equipo.

4. Capa de servicios centralizada

Todos los accesos a datos externos se abstraen en servicios independientes ubicados en `app/services/`. El `auth.service.ts` maneja el login y almacenamiento del token JWT. El `delivery.service.ts` gestiona el CRUD de entregas contra la API REST. El `client.service.ts` registra clientes con foto usando `multipart/form-data`. El

driver.service.ts registra repartidores con foto. El geocoding.service.ts provee autocomplete de direcciones via Nominatim. El connectivity.service.ts verifica el estado de red. El sqlite.service.ts gestiona la persistencia local de entregas.

Esta separación permite que los ViewModels no contengan lógica de red y que los servicios sean independientes entre sí.

5. Autenticación con JWT y ApplicationSettings

Se implementó autenticación stateless con JWT. Al iniciar sesión, el servidor devuelve un token JWT que el AuthService guarda en ApplicationSettings, que internamente usa SharedPreferences de Android. El rol del usuario se extrae del payload del JWT en ese mismo momento y se cachea para no repetir la operación de decodificación. Todas las peticiones posteriores incluyen el token en el header Authorization: Bearer.

6. Almacenamiento local con SQLite

Para el requisito de almacenamiento local se utilizó el paquete @nativescript-community/sqlite. La lógica implementada es: cuando hay conexión, se consulta la API REST, los datos se guardan en SQLite y se muestran en pantalla. Cuando no hay conexión o la API falla, se leen los datos de SQLite y se muestra un indicador de modo offline al usuario. La base de datos se almacena en el directorio de documentos del dispositivo con el nombre deliveries.db. Se eligió SQLite sobre ApplicationSettings porque permite almacenar colecciones estructuradas de forma eficiente con soporte para consultas ordenadas.

7. Autocomplete de direcciones con debounce

El autocomplete de direcciones en las pantallas de crear y editar entrega implementa varias medidas para evitar peticiones excesivas. Se aplica un debounce de 500ms que espera a que el usuario deje de escribir antes de llamar a la API. No se realiza ninguna llamada con menos de 3 caracteres escritos. Se verifica la conectividad antes de cada búsqueda y si el dispositivo está offline se muestra un mensaje inmediatamente. Si el texto cambió mientras la petición estaba en curso, el resultado se descarta para evitar condiciones de carrera. Los resultados se limitan a 5 sugerencias y se filtran exclusivamente a Costa Rica mediante el parámetro countrycodes=cr.

8. Manejo de imágenes con multipart/form-data

Para el registro de clientes y repartidores, las fotos de perfil se envían al servidor como parte de un formulario multipart/form-data usando el paquete @nativescript/background-http. Esta decisión se tomó porque el módulo estándar Http de NativeScript no soporta multipart nativo, y background-http permite enviar archivos junto con campos de texto en una sola petición. El usuario puede tomar la foto con la cámara del dispositivo o seleccionarla desde la galería.

9. Configuración centralizada de la API

Todas las URLs y endpoints se definen en un único archivo app/config/api.config.ts. Esto permite cambiar el entorno de desarrollo (emulador Android, dispositivo físico, producción) en un solo lugar sin modificar el código de los servicios. El archivo incluye comentarios que documentan la URL correcta para cada escenario de desarrollo.

10. Navegación con Frame de NativeScript Core

Se utilizó el sistema de navegación por Frame de NativeScript Core sin frameworks de UI adicionales, ya que el proyecto usa TypeScript puro. La navegación es imperativa desde los ViewModels mediante Frame.topmost().navigate() y Frame.topmost().goBack(). Esto mantiene la arquitectura simple y sin dependencias de frameworks externos como Angular Router.